

Apache Spark

ADAM BELLOUM

	
Developer(s)	Apache Software Foundation, UC Berkeley AMPLab, Databricks
Initial release	May 30, 2014; 18 months ago
Stable release	v1.5.2 / November 9, 2015; 51 days ago
Development status	Active
Written in	Scala, Java, Python, R
Operating system	Linux, Mac OS, Windows
Type	data analytics, machine learning algorithms
License	Apache License 2.0
Website	spark.apache.org



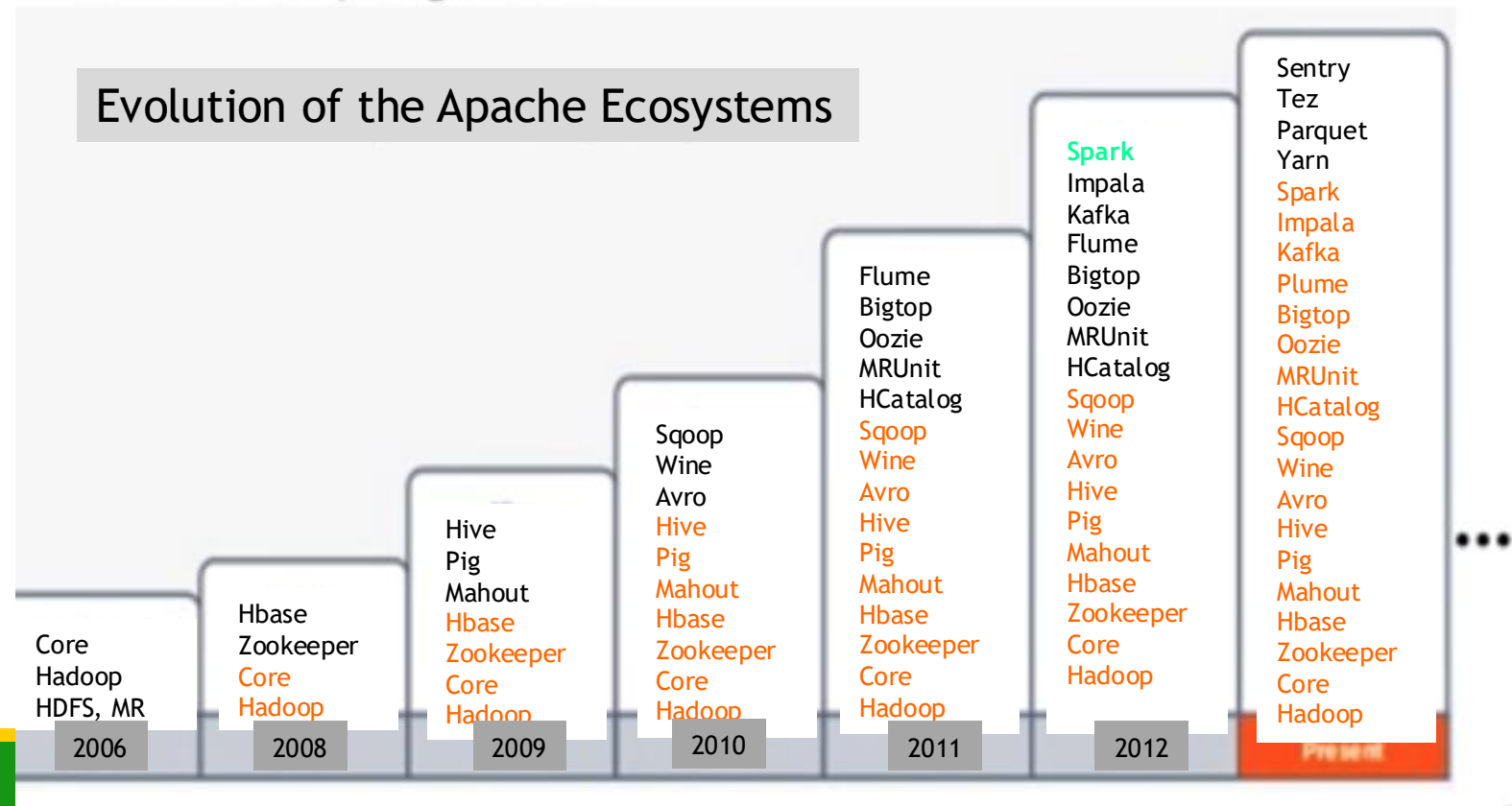
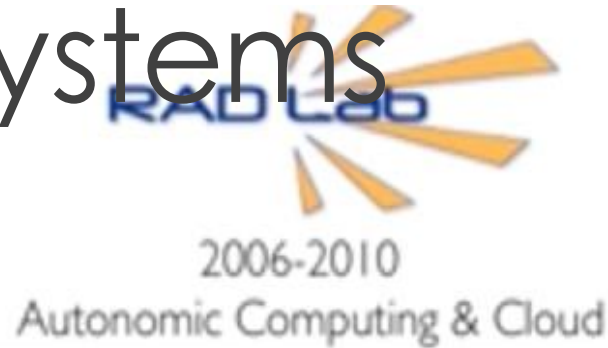
Funded by
the European Union

DIGITAfrica project has received funding from the EU Horizon Europe research and innovation Programme and Switzerland under Grant Agreement No. 101187966. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Executive Agency (REA). Neither the European Union nor the granting authority can be held responsible for them.



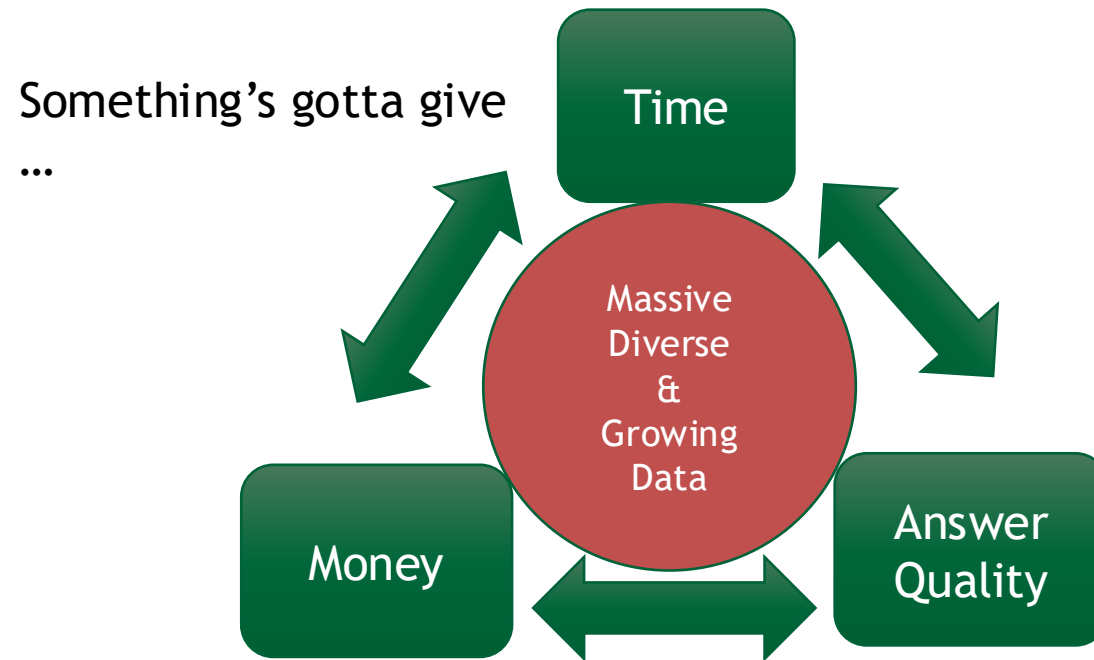


Evolution of the Apache Ecosystems





AmpLab view on BigData





Spark project goals

Generality:

- diverse Workload, operators and job sizes

Low Latency:

- sub-second

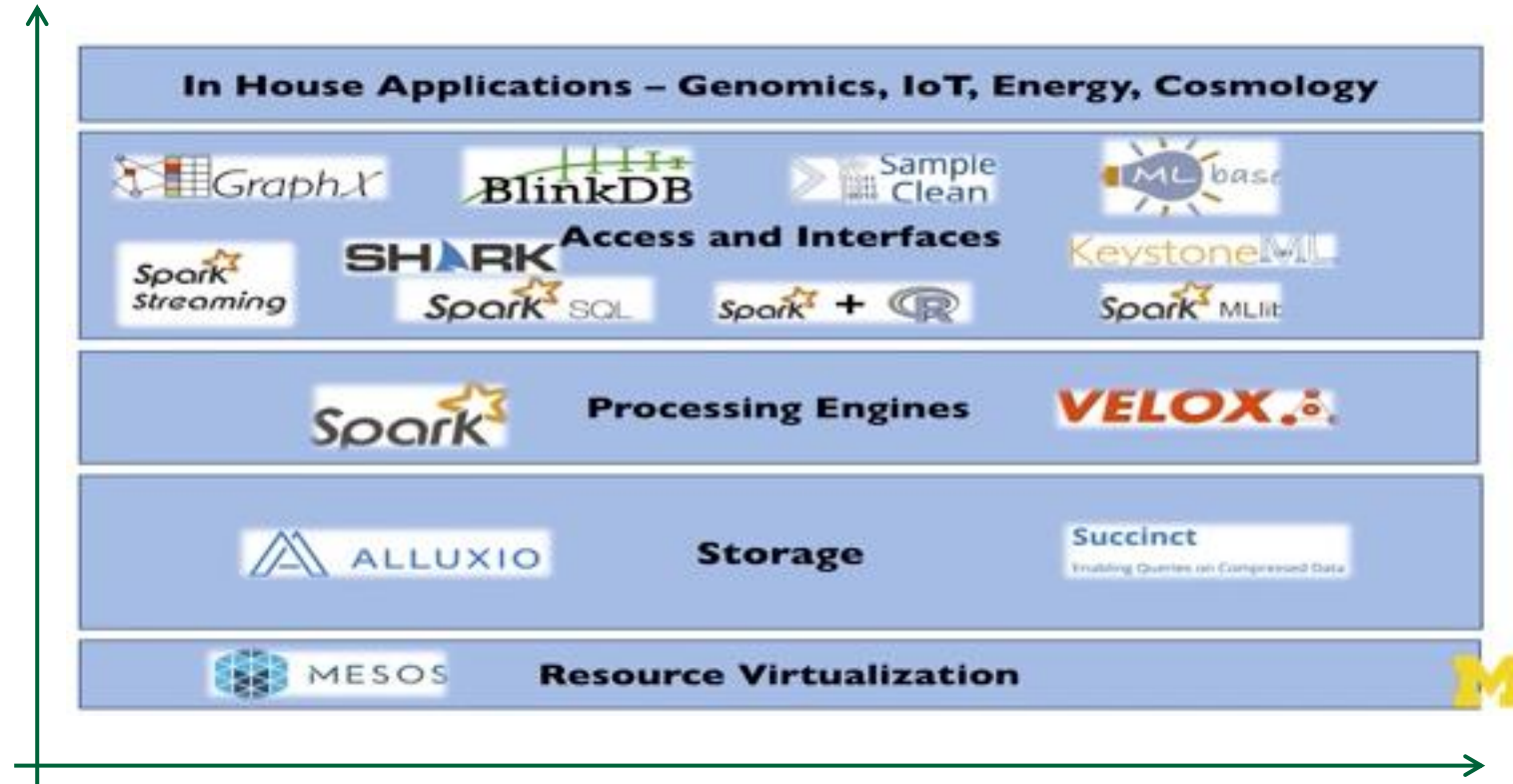
Fault tolerance:

- fault shouldn't be a special case

Simplicity



Berkeley Data Analytics Stack





Spark is generalisation of MR

MapReduce

Dremple, Giraph, Ampala, Drill, GraphLab, Storm, ...

- General Batch Specialized systems
- Processing (iterative, interactive, streaming Apps...)

With two additions to Hadoop MR, we **can enable more data processing** with single system

1. Richer programming model (General Task DAG)
2. Data Sharing (less data Movement)



What is Apache Spark

“Fast and general **Cluster** computing systems **interoperable** with **Hadoop**, included in all major distributions” Michael Armbrust

Improve efficiency through

- **In-memory** computing primitives
- General computation Graphs

Up to 100 x faster
(2 to 10 x on disk)

Improve **usability** through

- Rich APIs in Scala, Java, Python
- Interactive Shell

2 to 5 time less code



Spark Model

Write programs in Terms of **transformations** on **distributed datasets**

Resilient Distributed Datasets (RDD)

- Collection of objects that can be stored in **memory** or on **disk** across a cluster
- Parallel functional transformation (map, filter ,...)
- Automatically rebuilt on failure



Resilient Distributed Datasets (RDD)

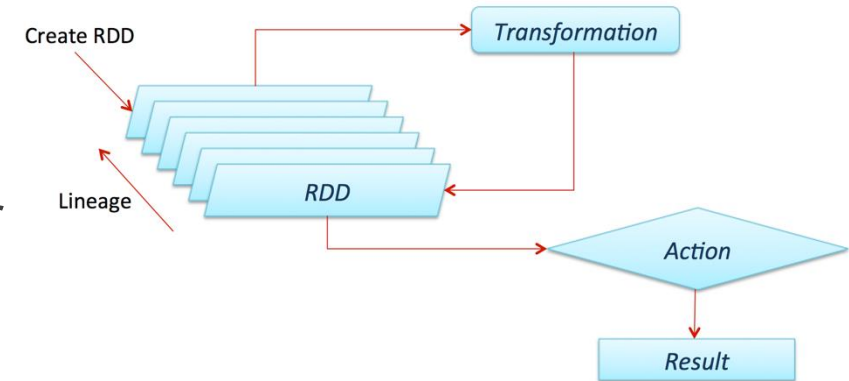
Partitioned collections of records

Created through **coarse-grained** deterministic transformations (map, filter, join...)

Immutable

Efficient fault-tolerance using **lineage**

- Log coarse-grained operations instead of fine-grained data updates
- An RDD has enough information about how it's derived from other dataset





More than Map & reduce

1. Richer programming model (General Task DAG)

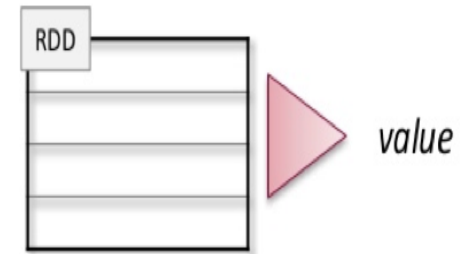
map	reduce	sample
filter	count	take
groupBy	fold	first
sort	reduceByKey	partitionBy
union	groupByKey	mapWith
join	cogroup	pipe
leftOuterJoin	cross	save
rightOuterJoin	zip	...
		



Operations on RDDs

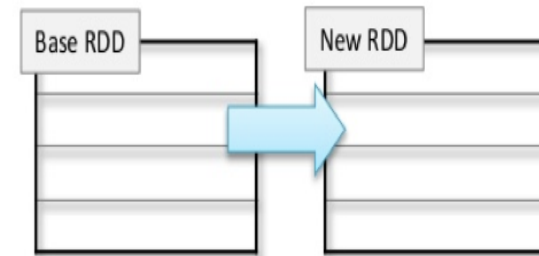
Actions:

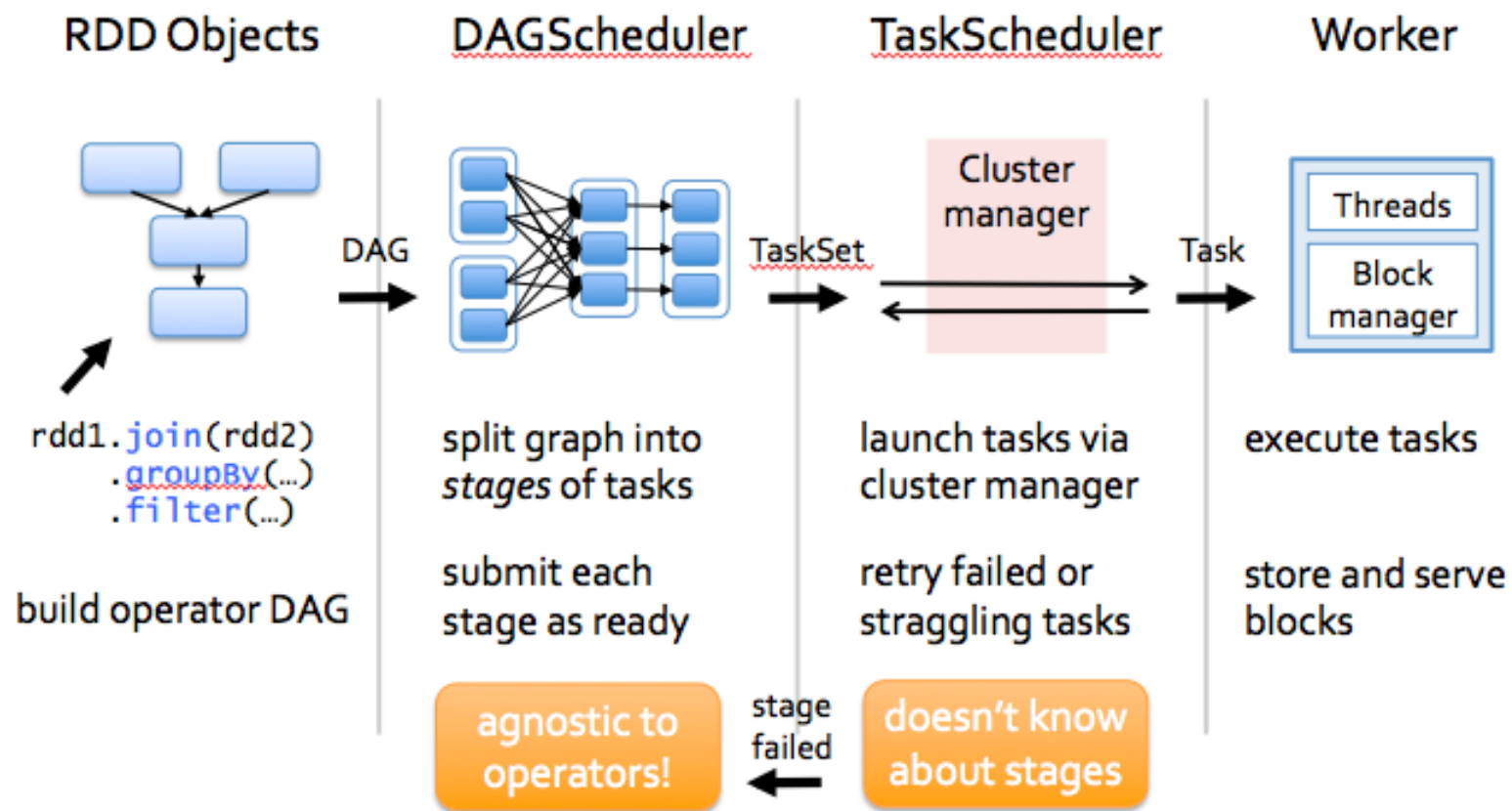
- Return some value or side-effect
- **Triggers computation**
- Example: 'count', 'saveAsTextFile'



Transformations:

- Create new RDD
- **Lazily** computed
- Example: 'map', 'filter'







Example: Log Mining

Load error messages from a log into memory then in reactively search for various patterns

```
val lines = spark.textfiles ("hdfs://..")
```

// lines → Base RDD

```
val errors = lines.filter(_.startswith "ERROR")
```

// errors → transformed RDD

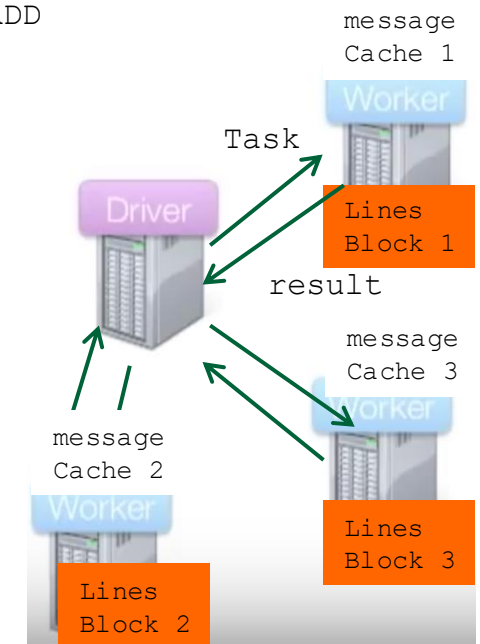
```
val messages = errors.map(_.split("\t") (2))
```

```
message.cache()
```

```
message.filter(_.contains "foo").count()
```

```
message.filter(_.contains "bar").count()
```

//Action



Result: Full-text search of Wikipedia in <1 sec
(vs 20 sec for on-disk data)

Result: scaled to 1 TB data in 5-7 sec
(vs 170 sec for on-disk data)



Word Count

INPUT

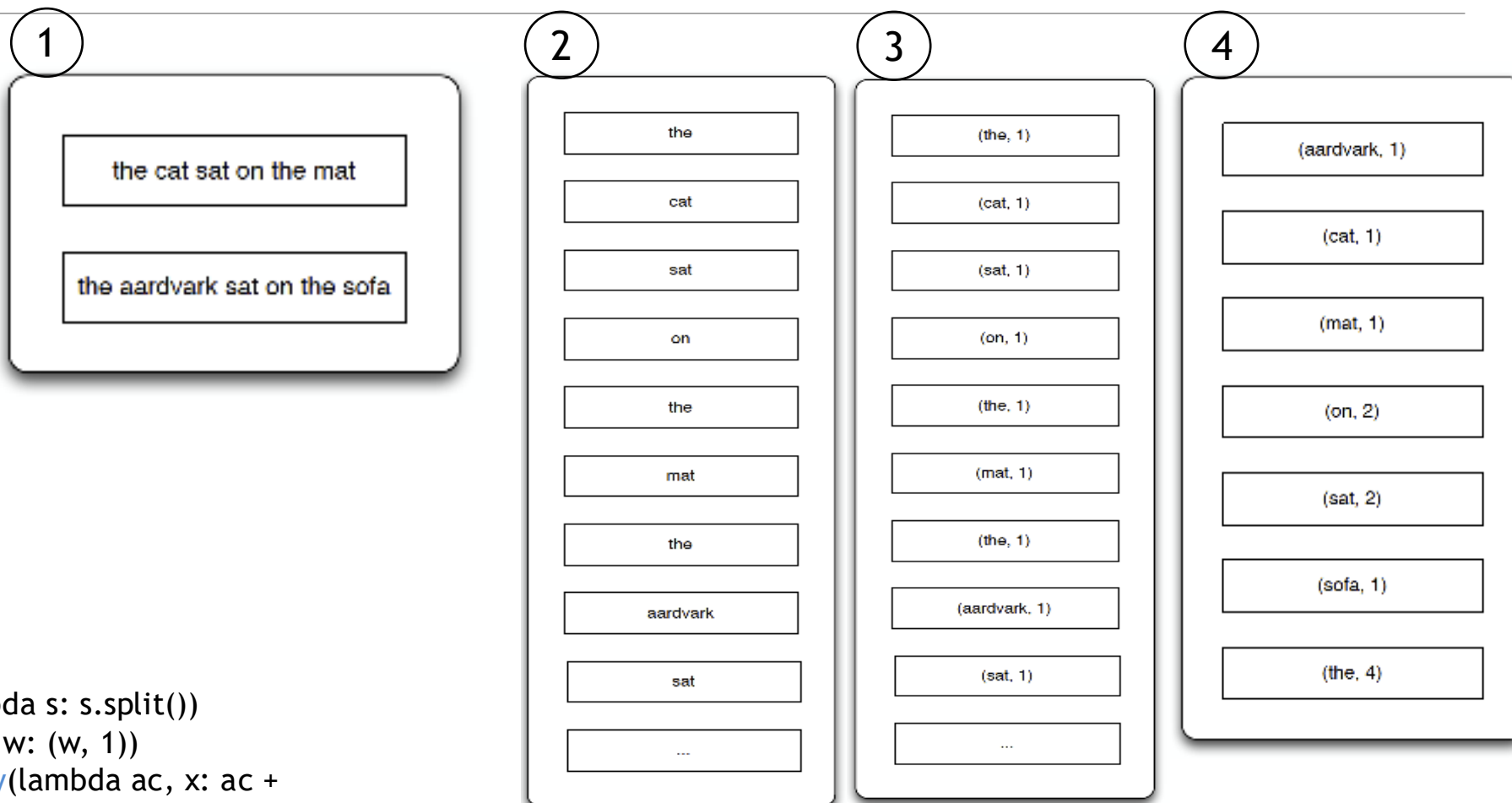
“the cat sat on the mat the
aardvark sat on the sofa”

OUTPUT

aardvark	1
cat	1
mat	1
on	2
sat	2
sofa	1
the	4



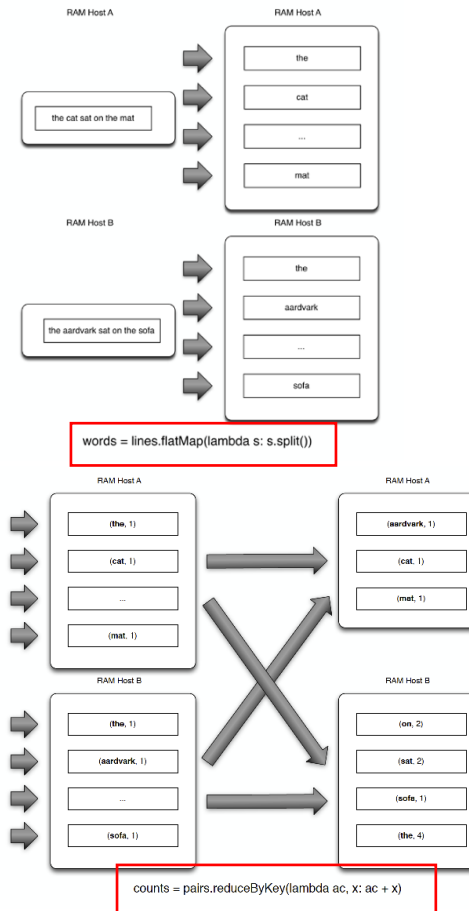
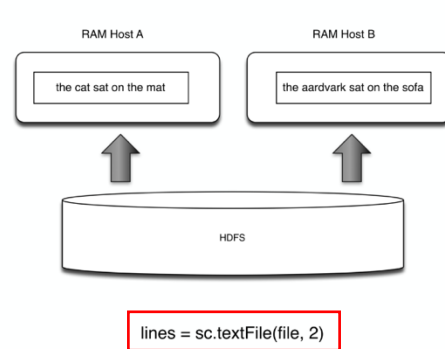
Word Count



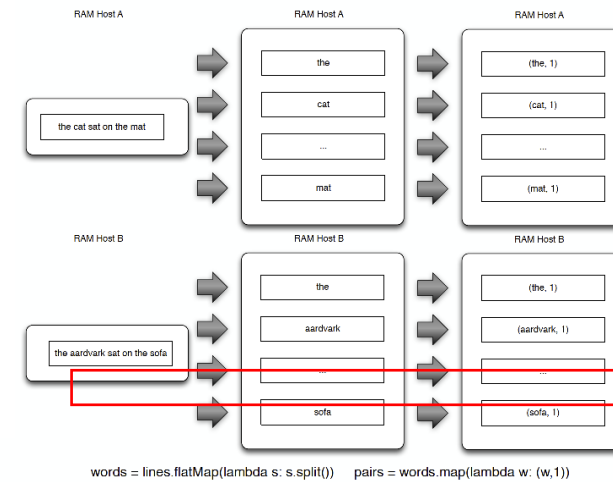
- ① `lines = sc.textFile(file)`
- ② `words = lines.flatMap(lambda s: s.split())`
- ③ `pairs = words.map(lambda w: (w, 1))`
- ④ `counts = pairs.reduceByKey(lambda ac, x: ac + x)`



Word Count



In-memory





Not just for in-memory data

WIRED

Startup Crunches 100 Terabytes of Data in a Record 23 Minutes

	Hadoop Record	Spark 100TB	Spark 1PB
Data Size	102.5TB	100TB	1000TB
Time	72 min	23 min	234 min
# Cores	50400	6592	6080
Rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Environment	Dedicate	Cloud (EC2)	Cloud (EC2)

amplab
UC BERKELEY

 databricks™



Performance

Spark normally faster but with caveats

- Spark **CAN** process data **in-memory**;
 - **Hadoop MapReduce** persists back to the disk after a map or reduce action
- Spark generally outperforms MapReduce, but it often needs **lots of memory to do well**; if there are other resource-demanding services or can't fit in memory, Spark degrades
 - **Hadoop MapReduce** runs alongside other services with minor performance differences,
 - **Hadoop MapReduce** works well with the 1-pass jobs it was designed for

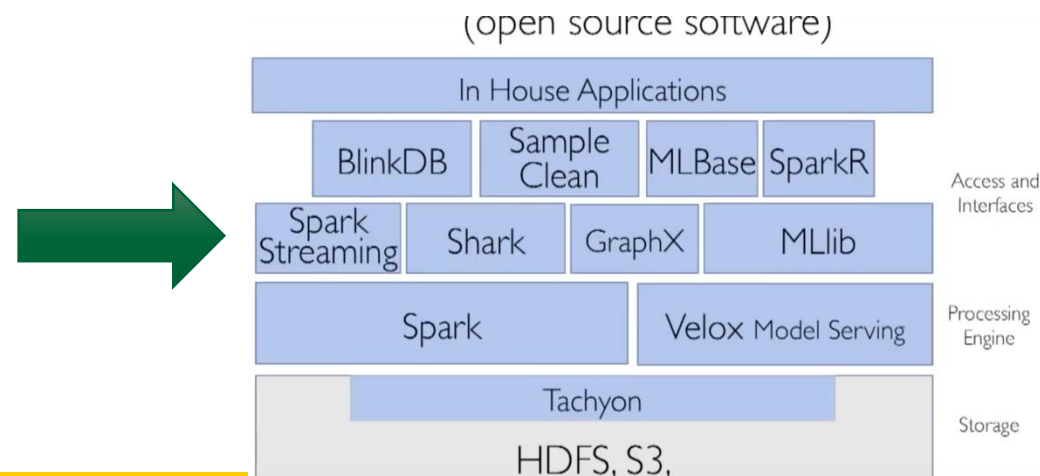
Ease of use: Spark is easier to program

Data processing: Spark more general

Maturity: Spark maturing, Hadoop MapReduce mature



Spark Streaming





Performance

Scales to **100s** of nodes

Can achieve **second scale** latencies

Integrates with Spark's batch and interactive processing

Provides a simple batch-like API for implementing complex algorithm

Can absorb live data streams from

- Kafka, Flume, ZeroMQ, etc.



What is Spark Stream

Spark Streaming is an extension of the core Spark API that enables scalable, high-throughput, fault-tolerant stream processing of live data streams





How Does Spark work?

1. Spark Streaming receives **live input data streams**
2. divides the data into **batches**,
3. which are then processed by the Spark engine to generate the final stream of results in batches





DStream

Dstream (*discretized stream*) is high-level abstraction, which represents a continuous stream of data.

DStreams can be created either from

- input data streams from sources such as Kafka, Flume, and Kinesis,
- or by applying high-level operations on other DStreams.

→ Internally, a DStream is represented as a **sequence of RDDs**



Word count from a Stream

Let's say we want to count the number of words in text data received from a data server listening on a TCP socket.

1. Import Spark Streaming Context
2. Create a Dstream *that will connect to hostname:port*
3. *Start processing the stream*
 - a. *Split the stream into words*
 - b. *Map*
 - c. *Reduce*



Word count using Spark Dstreams

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext
# Create a local StreamingContext with two working thread and batch interval of 1 second
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)
# Create a DStream that will connect to hostname:port, like localhost:9999
lines = ssc.socketTextStream("localhost", 9999)
# Split each line into words
words = lines.flatMap(lambda line: line.split(" "))
# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)
# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()
```

1.

2.

3. a.

3. b.

3. c.

Transformations on DStreams

map (*func*)

reduce (*func*)

flatMap (*func*)

filter (*func*)

transform (*func*)

updateStateByKey (*func*)

reduceByKey (*func*, [*numTasks*])

countByValue ()

Count ()

repartition (*numPartitions*)

union (*otherStream*)

join (*otherStream*, [*numTasks*])

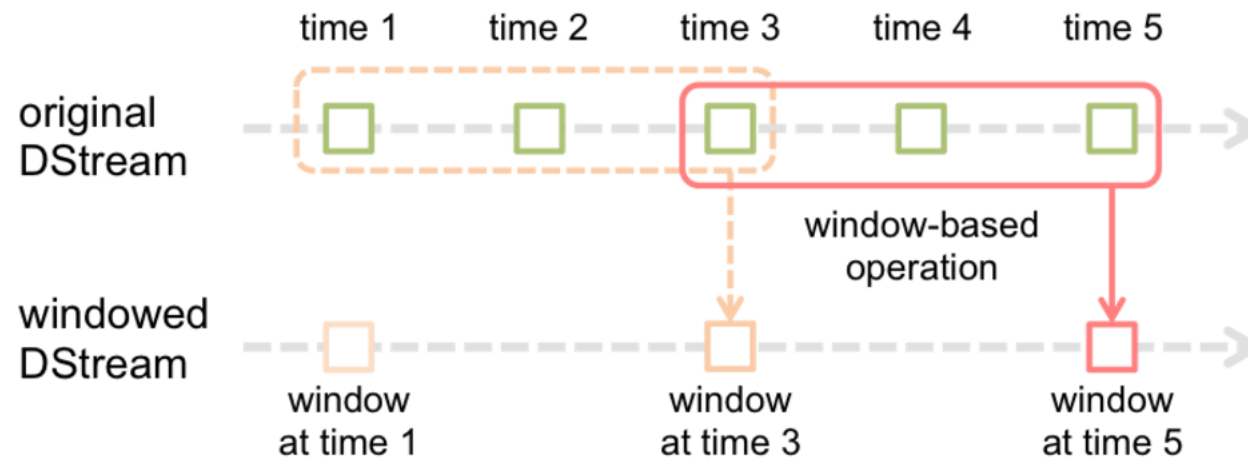
cogroup (*otherStream*, [*numTasks*])

Note: By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property `spark.default.parallelism`) to do the grouping. You can pass an optional `numTasks` argument to set a different number of tasks



Window Operations

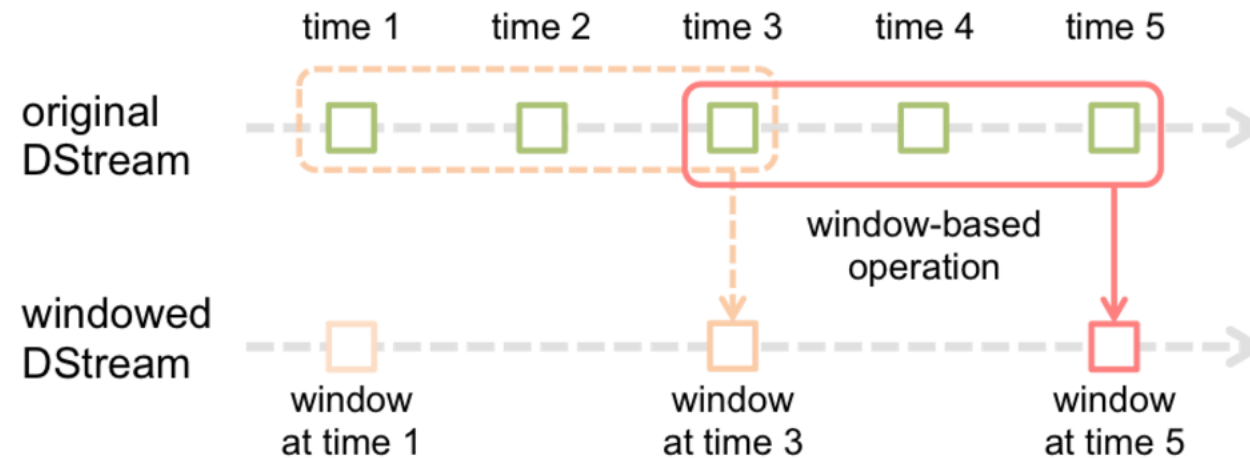
Spark Streaming also provides **windowed computations**, which allow you to apply transformations over a sliding window of data.





Window Operations

Every time the window slides over a source DStream, the source RDDs that fall within the window are combined and operated upon to produce the RDDs of the windowed DStream.



Transformations on DStreams

window(windowLength, slideInterval)

window(windowLength, slideInterval)

countByWindow(windowLength, slideInterval

reduceByKeyAndWindow(func, windowLength, slideInterval, [numTasks])

reduceByWindow(func, windowLength, slideInterval)

reduceByKeyAndWindow(func, invFunc, windowLength, slideInterval,
[numTasks])

countByValueAndWindow(windowLength, slideInterval, [numTasks])

Word count from a window on a stream



Let's say we want to count the number of words in text data received from a data server listening on a TCP socket. Now we want to count word over the last 30 seconds of data, every 10 seconds

1. Import Spark Streaming Context
2. Create a Dstream *that will connect to hostname:port*
3. *Start processing the stream*

Word count from a window on a stream



```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext
# Create a local StreamingContext with two working thread and batch interval of 1 second
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)
# Create a DStream that will connect to hostname:port, like localhost:9999
lines = ssc.socketTextStream("localhost", 9999)
# Split each line into words
words = lines.flatMap(lambda line: line.split(" "))
# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
# Reduce last 30 seconds of data, every 10 seconds
windowedWordCounts = pairs.reduceByKeyAndWindow(lambda x, y: x + y, lambda x, y: x - y, 30, 10)
# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()
```



Further readings

Joined UvA-VU Computer Science MSc Students Literature study reports:

Spark: Past, Present and Future

https://staff.fnwi.uva.nl/a.s.z.belloum/Project-Descriptions/Literature_Study/Baris-cem-viral-spark.pdf

Battle of the big data

https://staff.fnwi.uva.nl/a.s.z.belloum/Project-Descriptions/Literature_Study/Literature-study-tom-2.pdf